

# 3040 Self — Execution Plan v2 (Rebuild)

**For:** Google Antigravity (autonomous build agent) **Context:** This is a complete, standalone spec. You have no memory of prior attempts. Build only what is specified here, in the order specified.

---

## 0. What This App Is

A carbon footprint awareness web app. User logs trips (from/to/mode), the app computes carbon impact with a fixed lookup table (not AI), shows a mood-reactive character that reflects the user's rolling carbon trend, and uses Gemini to generate a short educational "fact card" per trip. 4-page React SPA, Firebase backend.

**v2 changes from v1 (why this version exists):** v1 included a 3D mascot (React Three Fiber), server-side Cloud Functions, Google Maps Places API, TTS/STT, and a time-triggered flashcard. All five were cut. Each was either too complex for reliable autonomous generation or too fragile to verify quickly. What remains is fully buildable with standard React + Firebase SDK calls — no custom backend, no 3D, no browser-quirk APIs.

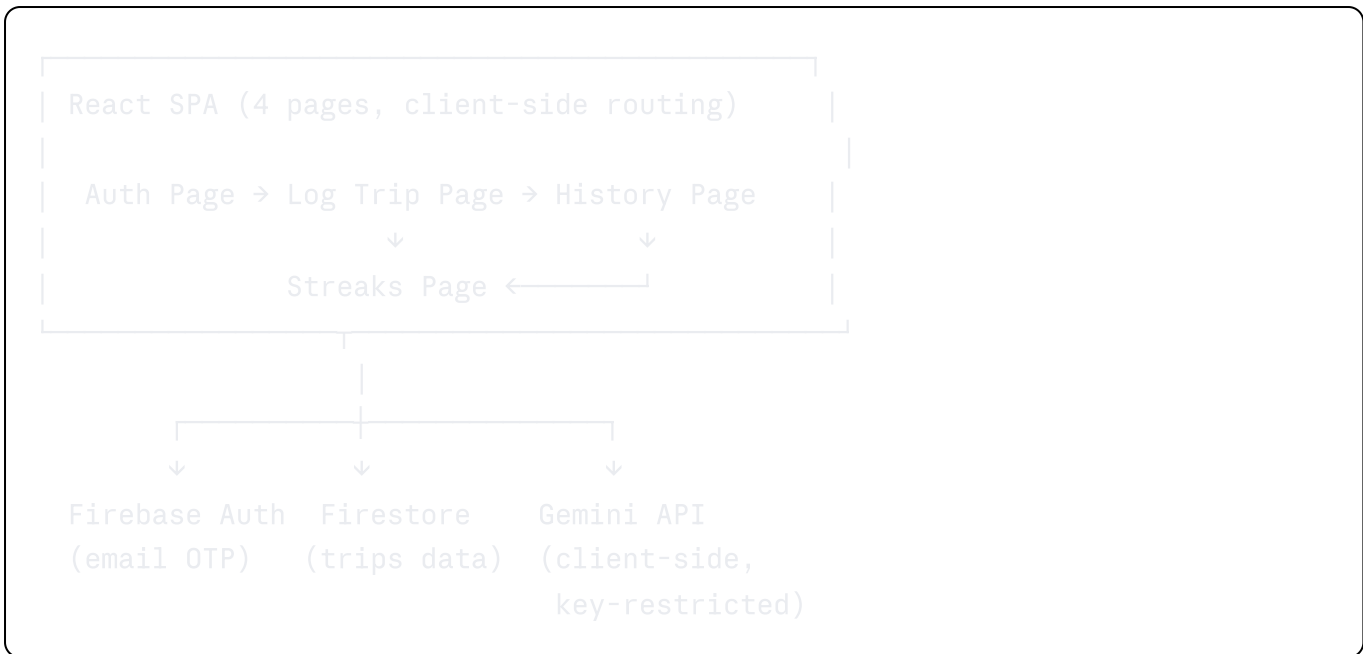
---

## 0.5 Repository & Submission Requirement

**Submission is a Git repo link. This is not optional polish — no repo, no submission.**

- Single root directory for the entire project (one repo, not nested/split repos). All Firebase config, frontend code, and this plan's structure live under one root.
  - Antigravity creates the GitHub repo (public, so it's viewable without auth) and pushes all code to it — this is a required final action, not implied by "the code exists locally."
  - Repo must include a `README.md` at root with: what the app does, how to run it locally, and which env vars/keys are needed (names only, never actual key values — see Security in Section 6).
  - `.gitignore` **must exclude:** `.env`, `node_modules/`, any Firebase service account JSON, any file containing the actual Gemini/Firebase API keys.
  - **Verification checkpoint (add to Section 4 as step 0):** Open the repo URL in an incognito/private browser tab — confirm code is visible, confirm no real API key is visible anywhere in any committed file (search the repo for the key string itself, not just check `.env`).
  - Final deliverable from Antigravity = the repo URL. State it explicitly when the build is done.
- 

## 1. Architecture



### Components + why:

- **Firebase Auth (email OTP)** — only auth method needed; no password storage/reset flow to build or break.
- **Firestore** — single `trips` collection, scoped by `userId`. No second database, no ORM, no schema migration risk.
- **Gemini API, client-side** — called directly from the browser with a key restricted by HTTP referrer (set in Google Cloud Console, not in code). This is a config step, not a code path, so it cannot be half-built. Real security tier below server-proxy, correct tradeoff for an agent with no backend deploy step.
- **Carbon math** — pure JS function, no network call, no AI. Lives in its own file, takes `(mode, distance)` → returns CO2 kg. This is the one piece of logic that must be 100% deterministic and 100% tested.
- **Mascot** — single React component, pure CSS (divs + border-radius + CSS keyframes), no SVG file, no image asset, no 3D library. Mood is a prop derived from rolling average of last N trips' carbon scores.

**No backend server. No Cloud Functions. No Maps SDK. No Web Speech API.**

---

## 2. Build Order

Order matters: wiring-critical and highest-failure-risk pieces first, so breakage shows immediately, not at the end.

1. **Firebase project setup + Auth (email OTP)** — nothing else works without a logged-in user. Verify first.
2. **Firestore trip write (manual form, 3 fields)** — verify a real document lands in Firestore before building anything that reads from it.

3. **Carbon math module (pure function)** — build and unit-test in isolation, before wiring into the UI.
  4. **Wire carbon math into trip submission** — on save, compute and store the CO2 value alongside the trip.
  5. **History page (Firestore read, paginated)** — first read-path; verify real data renders, not placeholder data.
  6. **Streaks page (derived from History data)** — pure function over already-verified data.
  7. **Mascot component (CSS-only, mood prop from rolling trend)** — depends on real trip data existing, build after #5.
  8. **Gemini fact-card (client-side call, key-restricted)** — last, because it's the one external network dependency outside Firebase; isolate failures here from everything else.
  9. **Splash screen + session persistence** — cosmetic, zero dependencies, do last.
- 

### 3. Per-Feature Spec

#### 3.1 Firebase Auth (email OTP)

- **Function:** User enters email, receives one-time code/link, signs in. Capture display name + persona choice (kid / friend / elder tone) at first sign-in, store on user profile doc.
- **Connections:** Firebase Auth SDK → on success, write `{name, persona, createdAt}` to `users/{uid}` in Firestore.
- **Fully working** = A real email is sent, code/link works, session persists on refresh, profile doc exists in Firestore with correct fields. Not "form renders" — "second browser tab with same login lands you in, logged in."
- **Rubric bar:** Security — no persona/name ever written to localStorage or URL params, only to Firestore under authenticated UID. Code Quality — auth state in one central hook/context, not re-checked ad hoc per component.
- **Verification checkpoint:** Sign in with a real email, close tab, reopen app, confirm still logged in. Check Firestore console — confirm `users/{uid}` doc exists with correct data.

#### 3.2 Manual Trip Logging

- **Function:** Form with exactly 3 inputs — From (text), To (text), Mode (dropdown: car / bus / train / bike / walk / flight). Submit writes to Firestore.
- **Connections:** Form → validates non-empty fields client-side → Firestore `trips` collection, doc includes `userId`, `from`, `to`, `mode`, `timestamp`.
- **Fully working** = Submitting writes a real document visible in Firestore console with all fields populated and correct `userId`. Form clears/confirms on success, shows real error on failure (not silently swallowed).

- **Rubric bar:** Efficiency — validate before the network call, not after a failed write. Accessibility — every input has a visible label (not placeholder-only), dropdown keyboard-navigable, error messages associated with their field via `aria-describedby`.
- **Verification checkpoint:** Submit a trip, immediately check Firestore console for the new doc. Try submitting with an empty field — confirm client-side block, no malformed write.

### 3.3 Carbon Math Module

- **Function:** Pure function `calculateCO2(mode, distanceKm) → co2Kg`, using a fixed coefficient lookup table (e.g. car: 0.21 kg/km, bus: 0.10, train: 0.04, bike: 0, walk: 0, flight: 0.25 — agent should source real published average coefficients, cite source in code comment).
- **Connections:** Pure function, no imports beyond the coefficient constant. Called by trip submission (3.4) only. Never calls AI, never makes a network request.
- **Fully working** = Function returns correct, deterministic output for every mode in the table. Same input always produces same output.
- **Rubric bar:** Testing — this function gets actual unit tests (one per mode, plus an invalid-mode edge case). Code Quality — coefficients in a named constant object, not magic numbers inline.
- **Verification checkpoint:** Run the unit tests. Confirm they exist and pass before this function is wired into anything else.

### 3.4 Wire Carbon Math into Trip Submission

- **Function:** On trip form submit, also capture/estimate distance (simplest viable v1: a 4th input, "approx distance in km," numeric) and call `calculateCO2`, storing `co2Kg` on the trip doc.
- **Connections:** Trip form → `calculateCO2()` → Firestore write, single transaction-like flow (compute before write, not after).
- **Fully working** = Every trip doc in Firestore has a populated, correct `co2Kg` field matching what the pure function would return for those inputs.
- **Rubric bar:** Code Quality — math logic stays in its own module, UI component only calls it, never duplicates the lookup table.
- **Verification checkpoint:** Log a trip with known mode + distance, manually calculate expected CO2, confirm Firestore doc matches.

### 3.5 History Page

- **Function:** List of past trips for the logged-in user, most recent first, paginated (limit 20 per page/load-more).
- **Connections:** Firestore query `where('userId', '==', uid).orderBy('timestamp', 'desc').limit(20)`.

- **Fully working** = Shows real logged trips, not mock data. Loading state while fetching, empty state if no trips, pagination actually fetches the next page (not the same 20 again).
- **Rubric bar:** Efficiency — `limit()` query, never `.get()` on full collection then filter client-side. Accessibility — list uses semantic `<ul>/<li>` or table markup, not divs with no structure.
- **Verification checkpoint:** Log 3 trips, refresh History — see exactly those 3, correct data, correct order.

### 3.6 Streaks Page

- **Function:** Calculate consecutive days with at least one logged trip; show current streak count.
- **Connections:** Pure function over the same trip data already fetched for History (don't re-query Firestore separately if avoidable — reuse).
- **Fully working** = Streak number is mathematically correct against actual logged dates, recalculates correctly after a new trip is logged today vs. a gap day.
- **Rubric bar:** Testing — streak calc as pure function, testable with mock date arrays (today logged + gap, consecutive days, single day, zero trips).
- **Verification checkpoint:** Manually walk through 3 test date sequences against the function output before trusting the UI display.

### 3.7 Mascot Component (CSS-only)

- **Function:** Visual character built from CSS shapes (divs, border-radius, box-shadow — no SVG file, no image), mood class swaps based on rolling average CO2 of last 5 trips vs. a threshold (low avg = "thriving" class, mid = "neutral," high = "struggling").
- **Connections:** Takes `mood` as a prop (computed from trip data, same data source as History/Streaks). Renders different CSS class → different colors/expression via CSS only. Idle animation via CSS `@keyframes` (e.g. gentle scale/bounce loop).
- **Fully working** = Mood visibly changes (not just on mount — re-renders when new trip data changes the rolling average), animation runs without re-mounting the whole component on every parent re-render.
- **Rubric bar:** Accessibility — `aria-label` states the mood in words (e.g. "Your future self is thriving") so it's not visual-only information. Code Quality — mood-to-class mapping is a small pure function, not inline conditional JSX sprawl.
- **Verification checkpoint:** Log trips that should push the rolling average into each of the 3 mood bands, confirm the character visibly changes class each time — not stuck on one mood.

### 3.8 Gemini Fact-Card

- **Function:** After a trip is logged, client calls Gemini API directly with trip details, gets back a short educational fact card relevant to that trip's mode/impact.

- **Connections:** Browser → Gemini API (key restricted via HTTP referrer, set in Google Cloud Console — not a code task, a console config step that must be done and confirmed before this is "secure"). Response rendered as a card on the trip confirmation / Study Corner.
- **Fully working** = Real Gemini response per trip, not a static/hardcoded string. Handles API failure gracefully (show a fallback message, don't crash or show a blank card).
- **Rubric bar:** Security — key restriction confirmed in Google Cloud Console (verification checkpoint below, not just "key exists"). Google Services — direct, real Gemini usage. Efficiency — don't call Gemini more than once per trip (no retry loops, no calling on every re-render).
- **Verification checkpoint:** (a) Log a trip, confirm a real, trip-specific fact card appears, not the same text every time. (b) In Google Cloud Console, confirm the API key has HTTP referrer restriction set to your deployed domain — screenshot this as proof for the submission.

### 3.9 Splash Screen + Session Persistence

- **Function:** Branded splash shown briefly on cold launch. Firebase auth state persists across reloads until explicit logout.
  - **Connections:** Firebase Auth's built-in persistence (default `LOCAL`) — no custom session/token logic to write.
  - **Fully working** = Closing and reopening the browser keeps the user logged in. Logout button actually clears the session (test by logging out, refreshing, confirming redirected to Auth page).
  - **Rubric bar:** trivial, low rubric weight — fast to build, don't over-invest time here.
  - **Verification checkpoint:** Logout, refresh, confirm logged out. Login, refresh, confirm still logged in.
- 

## 4. Verification Checkpoints — Full List (run in this order)

0. Repo: public GitHub URL exists, code visible in incognito tab, no real API keys found anywhere in committed files.
1. Auth: second tab / reopened browser stays logged in; Firestore user doc exists.
2. Trip write: Firestore console shows the real doc after submit; empty-field submit is blocked client-side.
3. Carbon math: unit tests exist and pass, independent of UI.
4. Trip + carbon math wired: Firestore doc's `co2kg` matches manual calculation.
5. History: real trips shown in correct order; pagination fetches new data, not a repeat.
6. Streaks: manually verified against 3 hand-checked date sequences.
7. Mascot: mood class visibly changes across logged trips, not static.

- 8. Gemini: fact card text differs per trip; API key restriction confirmed in Cloud Console with screenshot.
- 9. Session: logout/refresh and login/refresh both behave correctly.

No feature is "done" until its checkpoint is run and passed. A feature that renders but fails its checkpoint is a stub — flag it, don't ship it.

### 5. Risk Points (Claude + Antigravity, this build)

- **Gemini client-side call** is the single external network dependency with no Firebase SDK guardrails — most likely place for a silent failure (wrong endpoint, malformed request body, unhandled error response). Build this last, test it in isolation before integrating into the trip flow.
- **Firestore security rules** are not specified in this doc as code — agent must write rules restricting reads/writes to `request.auth.uid == resource.data.userId`. If this is skipped, it's a Security rubric failure even though the app "works." Flag explicitly as a required step, not optional hardening.
- **Rolling average for mascot mood** needs a minimum-trip-count guard (e.g. fewer than 3 trips logged = default/neutral mood, not a divide-by-zero or skewed result from 1 data point).
- **Distance input (3.4)** is a manual numeric field for v1 — if the agent tries to auto-add a Maps autocomplete here "to be helpful," stop it. Out of scope, explicitly cut.

### 6. Definition of Done — Rubric Mapping

| Rubric Category | Where it's satisfied                                                                                                                                                          |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code Quality    | Carbon math + streak calc as isolated pure functions; mood-mapping as small pure function; auth state in single context; no magic numbers (coefficients named constants)      |
| Security        | Firestore rules scoped to <code>request.auth.uid</code> ; Gemini key HTTP-referrer restricted (confirmed via Cloud Console screenshot); no sensitive data in localStorage/URL |
| Efficiency      | Firestore queries always <code>limit()</code> 'd, never full-collection scans; Gemini called once per trip, not on re-render; client-side validation before any write         |
| Testing         | Unit tests on carbon math (one per mode + edge case) and streak calc (3+ date scenarios) — these are the two functions with zero ambiguity, no excuse to skip                 |

| Rubric Category | Where it's satisfied                                                                                                                             |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Accessibility   | Labeled form inputs (not placeholder-only), keyboard-navigable dropdown, <code>aria-label</code> on mascot mood, semantic list markup on History |
| Google Services | Firebase Auth, Firestore, Gemini API — all real, all client-confirmed working per checkpoints above                                              |
|                 |                                                                                                                                                  |
|                 |                                                                                                                                                  |

7. Explicitly Out of Scope for v1 (stretch only, after all above verified)

- Maps-assisted trip entry (Places API)
- Voice input/output (Web Speech API)
- End-of-day "future self" letter flashcard
- 3D mascot / React Three Fiber

Do not build these until every checkpoint in Section 4 has passed. Reintroducing scope before the core is verified is how the last build ended up with stub buttons and disconnected APIs.